NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Departamento de Engenharia Electrotécnica e Computadores

Trabalho Final

Microprocessadores

Turno P4

10/11/2025

Alunos:

Catarina Afonso

Guilherme Gonçalves

Íris Gaspar

© 2020, DEEC/FCT/UNL

Índice

1. Introdução **3**

2. Descrição dos requisitos **4**

3. Apresentação da arquitectura implementada **5**

imagem 3.1 - Fluxograma jogo 6

4. Detalhe dos módulos do Sistema **8**

imagem 4.1 - Procedimento Menu 8

imagem 4.2 - Procedimento WaitAndClear 9

imagem 4.3 - Procedimento Grelha 10

imagem 4.4 - Procedimento SpawnFood 11

imagem 4.5 - Procedimento Updatebody 12

imagem 4.6 - Procedimento DrawScore 13

imagem 4.7 - SpawnHead => Crescimento da Cobra 13

imagem 4.8 - SpawnHead => Colisão com a grelha 13

5. Testes e Resultados **14**

imagem 5.1 - MENU 16

imagem 5.2 - Jogo 16

imagem 5.3 - Programadores 17

imagem 5.4 => GAME OVER 17

6. Conclusões **18**

© 2020, DEEC/FCT/UNL

1. INTRODUÇÃO

Este trabalho, proposto no âmbito da Unidade Curricular de Microprocessadores, tem como objetivo o desenvolvimento e implementação do Jogo da Cobra, utilizando a arquitetura x86-64 e o ambiente Win32 Console Application. O projeto visa aplicar os conhecimentos adquiridos sobre microprocessadores e programação em Assembly, através da criação de um jogo funcional que envolve movimentação, deteção de colisões e manipulação de estruturas de dados.

O Jogo da Cobra consiste numa cobra em movimento contínuo dentro de uma área delimitada, neste caso específico a área será de 80 x 25 caracteres, o animal é controlado pelo utilizador através de teclas de direção.

O objetivo deste jogo é comer o máximo de comida possível, que irá aparecer aleatoriamente no ecrã e aumentar o tamanho da cobra, quanto maior a cobra maior a pontuação. À medida que a cobra cresce, torna-se mais difícil evitar as limitações da grelha e a cobra em si, ao incidir nestas estruturas o jogo automaticamente acaba e o utilizador recebe a mensagem “GAME OVER” no ecrã, junto da sua pontuação final.

As principais funcionalidades implementadas foram a inicialização do ambiente de jogo (menu), a geração aleatória da comida e da direção inicial da cobra, o controlo da direção e do movimento da cobra, a deteção de colisões e a atualização da pontuação do utilizador.

Este relatório está estruturado em 8 capítulos:

- **Capítulo 1 -** Introdução;
- **Capítulo 2 -** Descrição das funcionalidades gerais do programa, incluindo as rotinas de inicialização, jogo e finalização;
- **Capítulo 3 -** Apresentação das principais funções implementadas, acompanhadas do respectivo código e explicações detalhadas;
- **Capítulo 4 -** Descrição das rotinas, com destaque para os parâmetros de entrada e saída;
- **Capítulo 5 -** Abordamento do funcionamento do programa final e dos resultados obtidos após os testes realizados;
- **Capítulo 6 -** Conclusão do trabalho;

© 2020, DEEC/FCT/UNL

2. Descrição dos requisitos

Este capítulo identifica os requisitos específicos do trabalho, detalhando as funcionalidades essenciais para a implementação do jogo proposto. O desenvolvimento concentrou-se em cinco áreas principais:

Exibição do Nome do Jogo e Identificação - De acordo com os requisitos, o programa inicia com um ecrã que apresenta o título "Jogo da Cobra" utilizando arte ASCII. Inclui a nossa identificação completa e aguarda uma tecla para iniciar. A implementação cumpre estes requisitos através do procedimento MENU, que mostra os nomes dos programadores (Guilherme Gonçalves, Catarina Afonso, Íris Gaspar) e as instruções de jogo.

Menu Inicial - O programa implementa um sistema de navegação conforme especificado, oferecendo três opções: 1 - Iniciar Jogo, 2 - Programadores, 3 - Sair.

Implementação do Jogo (Núcleo Principal) - A implementação cumpre todas as especificações técnicas:

- **Dimensões da grelha**: 80×25 caracteres (procedimento Grelha)
- **Caracteres especificados**: Parede '#', Cabeça '@', Corpo '+', Comida '*'
- **Estrutura da cobra**: Array de 200 segmentos (snakeX e snakeY) com SnakeDim controlado
- **Posicionamento inicial**: Centro do ecrã (calculado dinamicamente em SpawnHead)
- **Controlo por teclado**: WASD (Tanto maiúsculas como minúsculas).
- **Sistema de pontuação**: Score = Tamanho da cobra × Passos, conforme especificado
- **Deteção de colisões**: Verifica paredes (limites do ecrã) e auto-colisão
- **Crescimento**: Incremento de SnakeDim quando a cabeça atinge a comida

Highest Score - Implementação adicional que mantém e exibe o recorde da sessão, mostrando-o durante o jogo após o primeiro jogo completado. O sistema compara automaticamente a pontuação atual com high_score e atualiza quando necessário.

Ecrã de Game Over - Quando ocorre colisão, é apresentada a mensagem "GAME OVER", a pontuação final e o nome do utilizador, regressando posteriormente ao menu principal conforme especificado.

A implementação segue rigorosamente a arquitetura x86-64 para Windows Console Application, utilizando as APIs Win32 especificadas (GetStdHandle, WriteConsoleA, SetConsoleCursorPosition, Sleep, etc.) e respeitando todas as exigências técnicas do enunciado, desde a estrutura de dados até ao fluxo de jogo.

© 2020, DEEC/FCT/UNL

3. Apresentação da arquitectura implementada

O programa implementado segue uma arquitetura modular baseada em procedimentos em Assembly x86-64. O programa está organizado em torno do loop principal que coordena todas as operações.

Concebemos a estrutura para responder aos requisitos propostos e de forma a garantir uma execução fluida e eficiente do jogo. O sistema utiliza uma combinação de APIs do Windows para controlo da consola, funções da biblioteca C Runtime para operações de input/output e funções da biblioteca Legacy.

Para uma análise clara, o sistema foi dividido em três áreas funcionais principais.

3.1.1. Interface e Desenho (OUTPUT)

Esta área é responsável por renderizar todos os elementos visíveis no ecrã e gerir a sua apresentação visual.

Funções de Base:

- **GetStdHandle**: Essencial para obter o controlo (Handle) do ecrã (Saída Padrão);
- **SetConsoleCursorPosition**: Permite o posicionamento preciso de cada elemento gráfico nas coordenadas (X, Y) da consola;
- **WriteConsoleA**: Utilizada para a renderização rápida e eficiente de caracteres (parede, cobra, comida);
- **SetConsoleTextAttribute**: Permite a aplicação de cores (e.g., vermelho para o título, verde para a comida);
- **Beep**: Utilizada para melhorar a qualidade de jogo do utilizador usando efeitos sonoros quando o inicia, come e perde;
- **printf**: Utilizada para apresentar texto formatado (menus, *score*).

A renderização eficiente é possibilitada pelo uso da API do Windows.

Módulos Implementados:

- **Grelha**: Desenha o ambiente estático do jogo, definindo os limites através das paredes (#);
- **DrawScore**: Apresenta e atualiza a pontuação (**score** e **high_score**) e o nome do utilizador no topo do ecrã;
- **MENU**: Trata da apresentação inicial do jogo, incluindo os títulos, as opções e a identificação dos programadores;
- **ShowGameOver**: Apresenta o ecrã final com a pontuação obtida e mensagens de encerramento.

© 2020, DEEC/FCT/UNL

3.1.2. Lógica do Jogo

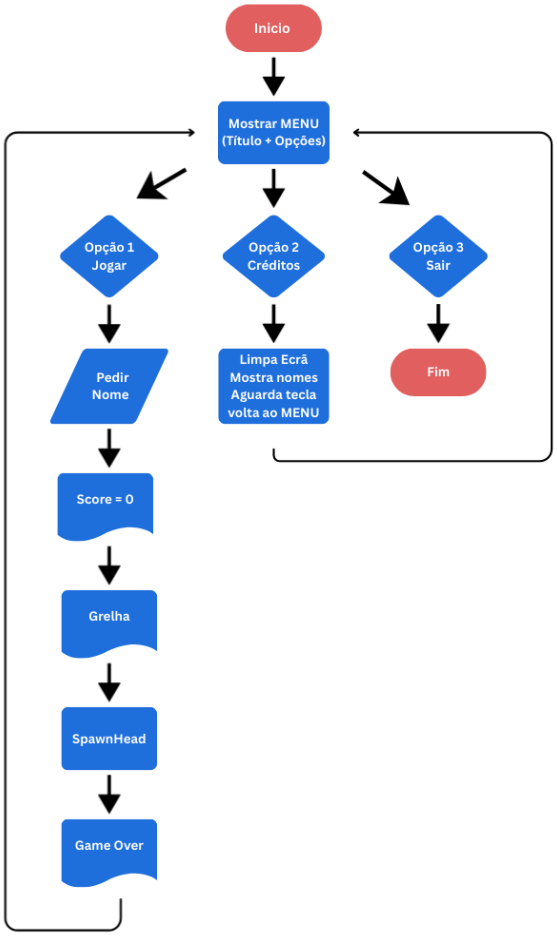
Esta área engloba o núcleo do jogo, as regras de funcionamento, o ciclo de atualização de estado e a gestão da memória da cobra.

Funções de Base:

- **Sleep**: Utilizada para pausar a execução por um período de tempo definido, controlando assim a velocidade do jogo (Frame Rate);
- **rdtsc / div**: Usadas indiretamente em **SpawnFood** para gerar números "aleatórios" para o posicionamento da comida.

Módulos Implementados:

- **SpawnHead**: É o núcleo principal (Game Loop), que gere o movimento da cabeça, a verificação de colisões (parede e auto-colisão) e a lógica de alimentação e crescimento.
- **UpdateBody**: Gere o movimento do corpo, copiando as coordenadas do segmento anterior para o segmento atual para simular o rastejar da cobra e assegurar o seu crescimento.
- **SpawnFood**: Implementa a lógica para a criação aleatória e o posicionamento da comida, garantindo que não aparece sobre a cobra ou nas paredes.
- **ResetGame**: Reinicia as variáveis de estado (score, SnakeDim) para um novo jogo.



© 2020, DEEC/FCT/UNL

Imagem 3.1 => Fluxograma jogo

3.1.3. Input e Sistema

Esta área gere a interação com o utilizador (teclado) e o controlo do ambiente de execução (sistema operativo).

Funções de Base:

- **_kbhit**: Permite a leitura não bloqueante do teclado (verifica se há teclas), o que é essencial para que o jogo continue a mover-se mesmo sem *input*;
- **_getch**: É utilizada para a leitura do input do jogador, seja para definir a direção no *loop* principal ou para confirmar opções no menu;
- **system**: Utilizada com o comando "cls" em **WaitAndClear** para limpar o ecrã, gerindo as transições entre o menu e o jogo;
- **scanf**: Utilizada em **ReadUserName** para ler o nome do utilizador.

A interação com o jogador é crucial para a jogabilidade em tempo real.

Módulos Implementados:

- **WaitAndClear**: Procedimento utilitário que gere a limpeza do ecrã para transições de interface;
- **ReadUserName**: Gere a interação inicial com o utilizador para obter o seu nome;
- **main**: O gestor do ciclo de vida do projeto, controlando o fluxo entre o menu, o início do jogo e o fim do programa.

Em suma, esta organização modular permitiu uma implementação clara e organizada, onde cada procedimento pode ser desenvolvido e testado independentemente, facilitando a manutenção e compreensão global do sistema. A abordagem demonstra como é possível estruturar código Assembly de forma legível e eficiente, cumprindo todos os objetivos funcionais estabelecidos para o projeto

© 2020, DEEC/FCT/UNL

4. Detalhe dos módulos do Sistema

O sistema, como já foi referido, é composto por vários módulos que cooperam para fornecer todas as funcionalidades do jogo, cada um com a sua especificidade e interface bem definida.

Menu - é responsável pela apresentação inicial do jogo, mostrando o título "JOGO DA COBRA" seguido dos nossos nomes. A implementação baseia-se em chamadas sucessivas à função printf para escrever cada linha no ecrã, criando uma apresentação simples mas eficaz.

```
234   ; ------------------------------------
235   ; MENU: Mostra opções e retorna escolha
236   ; ------------------------------------
237   MENU PROC
238       sub  rsp, 40
239       call WaitAndClear
240
241       ; --- Impressão da Arte ASCII ---
242       lea rcx, titulo0
243       call printf
244       lea rcx, titulo0
245       call printf
246
247       mov  ecx, STD_OUTPUT_HANDLE
248       call GetStdHandle
249       mov  rbx, rax
250
251       mov  rcx, rbx
252       mov  rdx, 12
253       call SetConsoleTextAttribute
254
255       lea  rcx, titulo1
256       call printf
257       lea  rcx, titulo2
258       call printf
259
260        mov  rcx, rbx
261       mov  rdx, 7                    ;
262       call SetConsoleTextAttribute
263
264
265       lea  rcx, titulo3
266       call printf
267       lea  rcx, titulo4
268       call printf
269
270       mov  rcx, rbx
271       mov  rdx, 12
272       call SetConsoleTextAttribute
273
274       lea  rcx, titulo5
275       call printf
276       lea  rcx, titulo6
277       call printf
278
279       mov  rcx, rbx
280       mov  rdx, 7
281       call SetConsoleTextAttribute
282
283       lea  rcx, titulo7
284       call printf
285       lea  rcx, espaco
286       call printf
287       lea  rcx, espaco
288       call printf
289
290       mov  rcx, rbx
291       mov  rdx, 12
292       call SetConsoleTextAttribute
293
294       lea  rcx, titulo10
295       call printf
296       lea  rcx, titulo11
297       call printf
298
299       mov  rcx, rbx
300       mov  rdx, 7
301       call SetConsoleTextAttribute
302
303       lea  rcx, titulo12
304       call printf
305       lea  rcx, titulo13
306       call printf
307
308       mov  rcx, rbx
309       mov  rdx, 12
310       call SetConsoleTextAttribute
```

© 2020, DEEC/FCT/UNL

```
312     lea  rcx, titulo14
313     call printf
314     lea  rcx, titulo15
315     call printf
316
317     mov  rcx, rbx
318     mov  rdx, 7
319     call SetConsoleTextAttribute
320
321     ; --- Impressão dos Nomes ---
322     lea  rcx, espaco
323     call printf
324     lea  rcx, espaco
325     call printf
326     lea  rcx, espaco
327     call printf
328     lea  rcx, espaco
329     call printf
330
331
332     ; --- Impressão das Opções ---
333     lea  rcx, menu_opt1
334     call printf
335     lea  rcx, menu_opt2
336     call printf
337     lea rcx, menu_opt3
338     call printf
340     lea rcx, como
341     call printf
342     lea rcx, jogo1
343     call printf
344     lea rcx, jogo2
345     call printf
346     lea rcx, jogo3
347     call printf
348     lea rcx, espaco
349     call printf
350
351     ; --- Ler Tecla ---
352     call _getch
353
354     add  rsp, 40
355     ret
356 MENU endp
```

Imagem 4.1 => Procedimento Menu

WaitAndClear - Limpa o ecrã utilizando o comando 'cls' do sistema através da função system. Este procedimento é utilizado para transições entre diferentes ecrãs do jogo, como a passagem do menu para o jogo principal ou entre ecrãs de informação.

```
358 ; ------------------------------------------------------------------------
359 ; WaitAndClear: Limpa o ecrã (versão simplificada)
360 ; ------------------------------------------------------------------------
361 WaitAndClear PROC
362     push rbp                        ; Guarda base pointer
363     mov  rbp, rsp                   ; Configura novo stack frame
364     sub  rsp, 32                    ; Aloca stack
365
366     ; Nota: O nome diz "Wait", mas aqui só estamos a limpar
367     ; porque o MENU já fez o "Wait" (espera) ao ler a tecla.
368     lea  rcx, cls_command           ; Comando "cls"
369     call system                     ; Limpa ecrã
370
371     add  rsp, 32                    ; Limpa stack
372     pop  rbp                        ; Restaura base pointer
373     ret                             ; Retorna
374 WaitAndClear endp
```

Imagem 4.2 => Procedimento WaitAndClear

© 2020, DEEC/FCT/UNL

Grelha - Renderiza o ambiente desenhando as paredes que delimitam a área de jogo de 80x25 caracteres conforme especificado nos requisitos. O módulo implementa um algoritmo de duplo ciclo que percorre sistematicamente todas as posições da grelha, decidindo em cada posição se deve desenhar o caractere '#' para representar as paredes ou um espaço vazio para o interior do campo de jogo.A escrita é otimizada através da função WriteConsoleA da API Windows, que permite uma renderização mais rápida comparativamente à utilização de printf.

```
640   ; ---------------------------------
641   ; GRELHA: Desenha as paredes (#)
642   ; ---------------------------------
643   Grelha PROC
644       sub  rsp, 40
645       mov  ecx, STD_OUTPUT_HANDLE
646       call GetStdHandle
647       mov  rbx, rax
648       xor  rsi, rsi
649
650   next_row:
651       mov  rax, rsi
652       cmp  rax, [box_height]
653       jge  fim
654       xor  rdi, rdi
655
656   next_col:
657       mov  rax, rdi
658       cmp  rax, [box_width]
659       jge  print_newline
660
661       ; Lógica de Borda
662       cmp  rsi, 0
663       je   desenha_wall
664       mov  rax, [box_height]
665       dec  rax
666       cmp  rsi, rax
667       je   desenha_wall
668       cmp  rdi, 0
669       je   desenha_wall
670       mov  rax, [box_width]
671       dec  rax
672       cmp  rdi, rax
673       je   desenha_wall
674       jmp  desenha_space
676   desenha_wall:
677       mov  rcx, rbx
678       lea  rdx, wall_char
679       mov  r8, 1
680       lea  r9, [rsp+8]
681       call WriteConsoleA
682       jmp  continue_col
683
684   desenha_space:
685       mov  rcx, rbx
686       lea  rdx, space_char
687       mov  r8, 1
688       lea  r9, [rsp+8]
689       call WriteConsoleA
690   continue_col:
691       inc  rdi
692       jmp  next_col
693
694   print_newline:
695       mov  rcx, rbx
696       lea  rdx, newline
697       mov  r8, 2
698       lea  r9, [rsp+8]
699       call WriteConsoleA
700       inc  rsi
701       jmp  next_row
702   fim:
703       add  rsp, 40
704       ret
705   Grelha ENDP
```

Imagem 4.3 => Procedimento Grelha

© 2020, DEEC/FCT/UNL

SpawnFood - Gera comida utilizando o contador de tempo do processador através da instrução rdtsc para criar valores pseudo-aleatórios. Estes valores são processados para gerar coordenadas X entre 1 e 78 e coordenadas Y entre 3 e 23, garantindo que a comida aparece dentro dos limites jogáveis e não sobre as paredes. A comida é representada pelo caractere '*' e é imediatamente renderizada na posição calculada utilizando as mesmas funções de renderização da grelha.

```
707  ; ------------------------------------------
708  ; SpawnFood: Gera comida em posição aleatória
709  ; ------------------------------------------
710  SpawnFood PROC
711      push rbx
712      push rax
713      push rcx
714      push rdx
715      sub  rsp, 32
716
717      mov  ecx, STD_OUTPUT_HANDLE
718      call GetStdHandle
719      mov  rbx, rax
720
721      ; Gerar X (1-78)
722      rdtsc
723      xor  rdx, rdx
724      mov  rcx, 78
725      div  rcx
726      add  rdx, 1
727      mov  [foodX], rdx
728
729      ; Gerar Y (3-23)
730      rdtsc
731      xor  rdx, rdx
732      mov  rcx, 21
733      div  rcx
734      add  rdx, 3
735      mov  [foodY], rdx
736
737      ; Desenhar
738      mov  rax, rdx
739      shl  rax, 16
740      mov  rdx, [foodX]
741      or   ax, dx
742      mov  rdx, rax
743      mov  rcx, rbx
744      call SetConsoleCursorPosition
746      mov rcx, rbx
747      mov rdx, 10
748      call SetConsoleTextAttribute
749
750      mov  rcx, rbx
751      lea  rdx, food_char
752      mov  r8, 1
753      lea  r9, [rsp+8]
754      call WriteConsoleA
755
756      mov rcx, rbx
757      mov rdx, 7
758      call SetConsoleTextAttribute
759
760
761      add  rsp, 32
762      pop  rdx
763      pop  rcx
764      pop  rax
765      pop  rbx
766      ret
767  SpawnFood ENDP
```

Imagem 4.4 => Procedimento SpawnFood

© 2020, DEEC/FCT/UNL

UpdateBody - Responsável pela movimentação do corpo da cobra, implementando a lógica de que cada segmento deve ocupar a posição do segmento anterior. O algoritmo percorre os arrays snakeX e snakeY do último segmento em direção ao segundo, copiando as coordenadas da posição anterior para a atual. Esta abordagem assegura que todo o corpo segue fielmente o trajecto da cabeça, criando o movimento característico da cobra.

```
167   ; ---------------------------------------------------------------------------
168   ; UpdateBody: Move o corpo (copia índice i-1 para i)
169   ; ---------------------------------------------------------------------------
170   UpdateBody PROC
171       mov rcx, [SnakeDim]                ; Contador = Tamanho da cobra
172
173   shift_loop:
174       cmp rcx, 0                         ; Verifica se chegou ao pescoço (0)
175       je  fim_update                     ; Se sim, acabou
176
177       mov rdx, rcx                       ; Copia índice atual
178       dec rdx                            ; RDX = Índice anterior (i-1)
179
180       ; --- Mover X ---
181       lea r8, snakeX                     ; Endereço do array X
182       mov rax, [r8 + rdx*8]              ; Lê valor da posição anterior
183       mov [r8 + rcx*8], rax              ; Escreve na posição atual
184
185       ; --- Mover Y ---
186       lea r8, snakeY                     ; Endereço do array Y
187       mov rax, [r8 + rdx*8]              ; Lê valor da posição anterior
188       mov [r8 + rcx*8], rax              ; Escreve na posição atual
189
190       dec rcx                            ; Decrementa contador
191       jmp shift_loop                     ; Repete
192
193   fim_update:
194       ret                                ; Retorna
195   UpdateBody ENDP
```

Imagem 4.5 => Procedimento Updatebody

DrawScore - Desenha a pontuação no topo do ecrã, mostrando o nome do jogador e o score atual. Após o primeiro jogo, exibe também o highest score, que é atualizado automaticamente quando o jogador bate o seu próprio recorde. A lógica baseia-se num contador de jogos: inicialmente não mostra o recorde (0), mas após o primeiro jogo completo passa a exibir e comparar a pontuação com o valor guardado em memória.

© 2020, DEEC/FCT/UNL

```
197  ; ---------------------------------------
198  ; DrawScore: Desenha a pontuação no topo
199  ; ---------------------------------------
200  DrawScore PROC
201      push rbx
202      sub  rsp, 32
203
204      mov  ecx, STD_OUTPUT_HANDLE
205      call GetStdHandle
206      mov  rbx, rax
207      xor  rax, rax
208      mov  rdx, rax
209      mov  rcx, rbx
210      call SetConsoleCursorPosition
211
212      lea rcx, utilizadorjogo
213      lea rdx, utilizador_buffer
214      call printf
215
216      lea  rcx, score_format
217      mov  rdx, [score]
218      call printf
219
216      lea  rcx, score_format
217      mov  rdx, [score]
218      call printf
219
220      cmp [contador_de_jogos], 0
221      je  fim
222
223      lea rcx, high_score_print
224      mov rdx, [high_score]
225      call printf
226
227      fim:
228      add  rsp, 32
229      pop  rbx
230      ret
231
232  DrawScore ENDP
```

Imagem 4.6 => Procedimento DrawScore

SpawnHead - Implementa o loop principal do jogo e coordena todas as operações em cada frame. Este módulo inicia por atualizar as posições do corpo da cobra através de UpdateBody. Segue-se o movimento da cabeça segundo a direção atual armazenada nas variáveis dX e dY, e a verificação de colisões com as paredes e com o próprio corpo. Quando é detectado que a cabeça alcançou a comida, a pontuação é incrementada, o tamanho da cobra aumenta e uma nova comida é gerada. O processamento de input do utilizador é feito de forma não-bloqueante, permitindo alterar a direção mediante as setas do teclado, com proteção contra inversões de direção que causariam colisão imediata.

```
516      ; --- Verificar Comida ---
517      mov  rax, [foodX]
518      cmp  r12, rax
519      jne  check_walls
520      mov  rax, [foodY]
521      cmp  r13, rax
522      jne  check_walls
523
524      ; --- Comeu a Fruta ---
525      call DrawScore
526      add  [SnakeDim], 1
527      call SpawnFood
```

Imagem 4.7 => SpawnHead => Crescimento da Cobra

```
529  check_walls:
530      ; CIMA (Y < 2)
531      cmp  r13, 3
532      jl   game_over
533      ; LADOS
534      mov  rax, [box_width]
535      dec  rax
536      cmp  r12, rax
537      jge  game_over
538      cmp  r12, 1
539      jle  game_over
540      ; BAIXO (Y > 25)
541      mov  rax, [box_height]
542      cmp  r13, rax
543      jg   game_over
```

Imagem 4.8 => SpawnHead => Colisão com a grelha

© 2020, DEEC/FCT/UNL

5. Testes e Resultados

A implementação do Jogo da Cobra foi validada através de uma série de testes funcionais, organizados em fases evolutivas, conforme o desenvolvimento de cada módulo. O procedimento de testes centrou-se em garantir a conformidade com os requisitos de jogabilidade.

5.1. Procedimentos das Fases:

Setup Inicial:

- Impressão do Nome dos Programadores;
- Construção da Grelha, **Grelha**;
- Construção da Cabeça, **SpawnHead**;

Movimento Básico:

- Atualização de X e Y usando **SpawnHead** e o delay da velocidade, **Sleep**;
- Colisão com Parede (#), morrer ao tocar neste caracter, verificando usando **SpawnHead**;
- Colisão com Corpo (+), verificação de r12 ,r13 com snakeX, snakeY no **SpawnHead**;

Pontuação Base:

- Pontuação 1 por cada passo dado, imprimida por **DrawScore**;

Direção Inicial:

- Direção Inicial Fixa seguida pela Direção Inicial Variável, usando **rdtsc** para definir o vetor aleatório;

Comida/Crescer:

- Randomize de Comida, **SpawnFood**;
- Crescimento, detectar colisão com (*), adicionar ao array SnakeDim, 1 (add [SnakeDim], 1), junto da execução da função **UpdateBody**;

Menu:

- Saída do Programa usando **exit:proc**;

Pontuação Final:

- Pontuação Nova = Tamanho da Cobra * Passos;

Nome dos Utilizadores:

- Solicitação do Nome do Utilizador sempre que é iniciado uma nova partida, usando **ReadUserName**, é impresso em **DrawScore** usando a string **utilizadorjogo** e **ShowGameOver**, para imprimir a última mensagem, usando a string **go_user**.

Estética Final:

© 2020, DEEC/FCT/UNL

- Cores, separação das áreas do Menu e estética final, **ShowGameOver**.

5.2. Testes

Para testar o nosso código, seguimos uma abordagem muito prática e faseada. Cada vez que acabávamos uma parte do código, compilamos logo e executávamos para ver se funcionava. Usámos o ml64 para compilar e depois corríamos o .exe diretamente.

Começámos por testar as partes mais simples primeiro. Compilámos só o menu para ver se os nossos nomes apareciam bem e se as opções (1, 2, 3) respondiam quando carregávamos nas teclas. Depois testámos a grelha sozinha, para garantir que as paredes desenhavam nas posições certas. Só quando isto estava bem é que passámos para a parte mais complicada.

Para testar a cobra, primeiro fizemos andar só a cabeça sem corpo, para ver se o movimento funcionava nas quatro direções. Depois adicionámos o corpo e testámos se seguia bem a cabeça. Para testar as colisões, fizemos de propósito bater nas paredes e em nós mesmos para ver se o jogo acabava como devia.

A comida foi testada aos poucos. Primeiro vimos se aparecia em lugares aleatórios, depois se a cobra conseguia comer e, finalmente, se crescia mesmo quando comia. A pontuação testámos contando os passos e o tamanho da cobra e comparando com o que aparecia no ecrã.

Sempre que algo não funcionava, olhávamos para o código da parte que tínhamos acabado de fazer. Muitas vezes eram erros pequenos, como comparar números mal ou não guardar valores nos sítios certos da memória. O mais difícil foi fazer o crescimento da cobra funcionar bem, porque tinha de crescer logo no momento certo e ficar no sítio certo.

Testámos também as cores e o aspecto final, mudando entre vermelho, verde e branco para ver se ficava bonito. O highest score testámos jogando várias vezes seguidas para ver se guardava mesmo o recorde mais alto.

5.3. Resultados Finais

Depois de termos todas as partes do jogo a funcionar individualmente, fizemos os testes finais para garantir que tudo corria bem junto. Jogámos o jogo completo várias vezes, com diferentes estratégias: umas vezes tentando durar o máximo tempo possível, outras tentando crescer o mais rápido possível.

Verificámos que o fluxo completo do jogo funcionava sem problemas: começava no menu, pedia o nosso nome, desenhava a grelha, a cobra movia-se, comia, crescia, e quando morria, mostrava o ecrã de game over e voltava ao menu. Testámos também os casos menos comuns, como tentar sair do jogo a meio (não era possível, tínhamos de deixar a cobra morrer) ou carregar em teclas que não eram para o jogo.

Confirmámos que a pontuação final estava sempre certa, que o highest score se atualizava quando devia, e que o nosso nome aparecia tanto durante o jogo como no final. Testámos em computadores diferentes para garantir que as cores e o tamanho do ecrã estavam sempre bem.

Estes testes finais mostraram que o nosso Jogo da Cobra estava pronto e funcionava tal como

© 2020, DEEC/FCT/UNL

tínhamos planeado.



imagem 5.1 => MENU

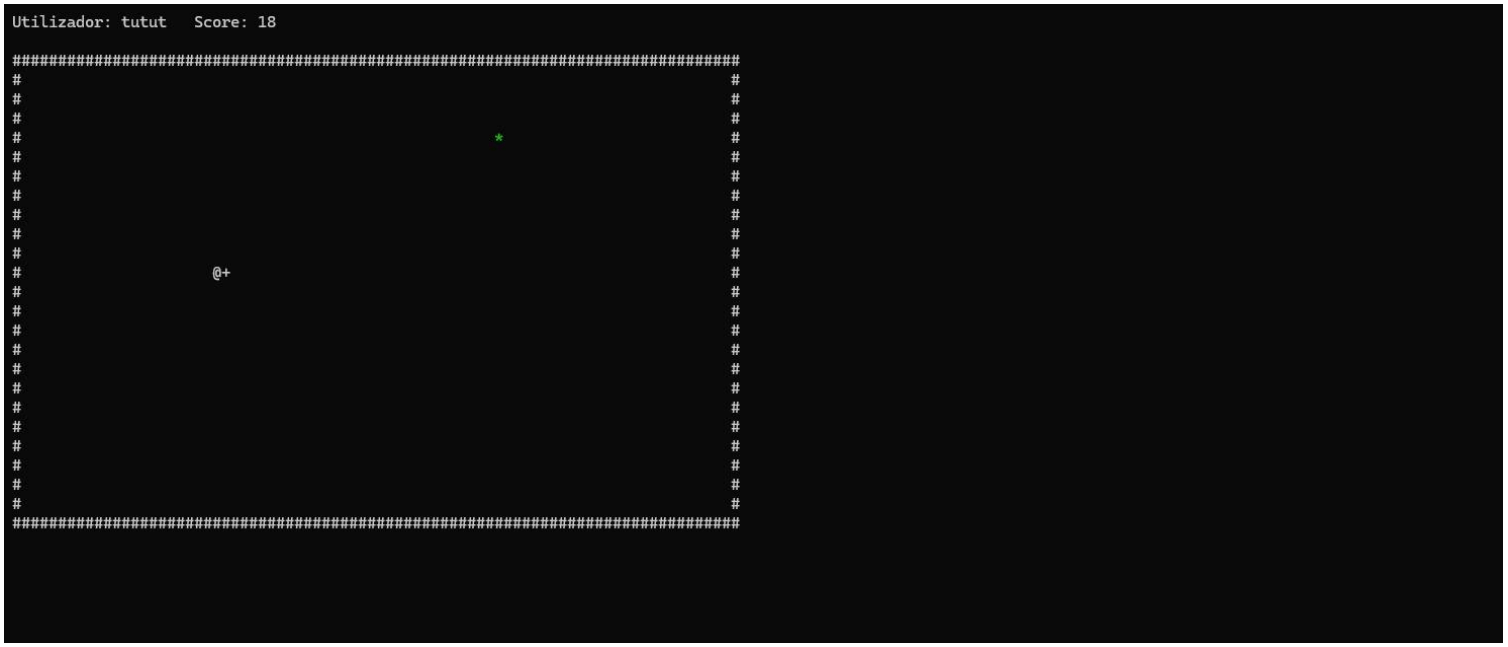


imagem 5.2 => Jogo

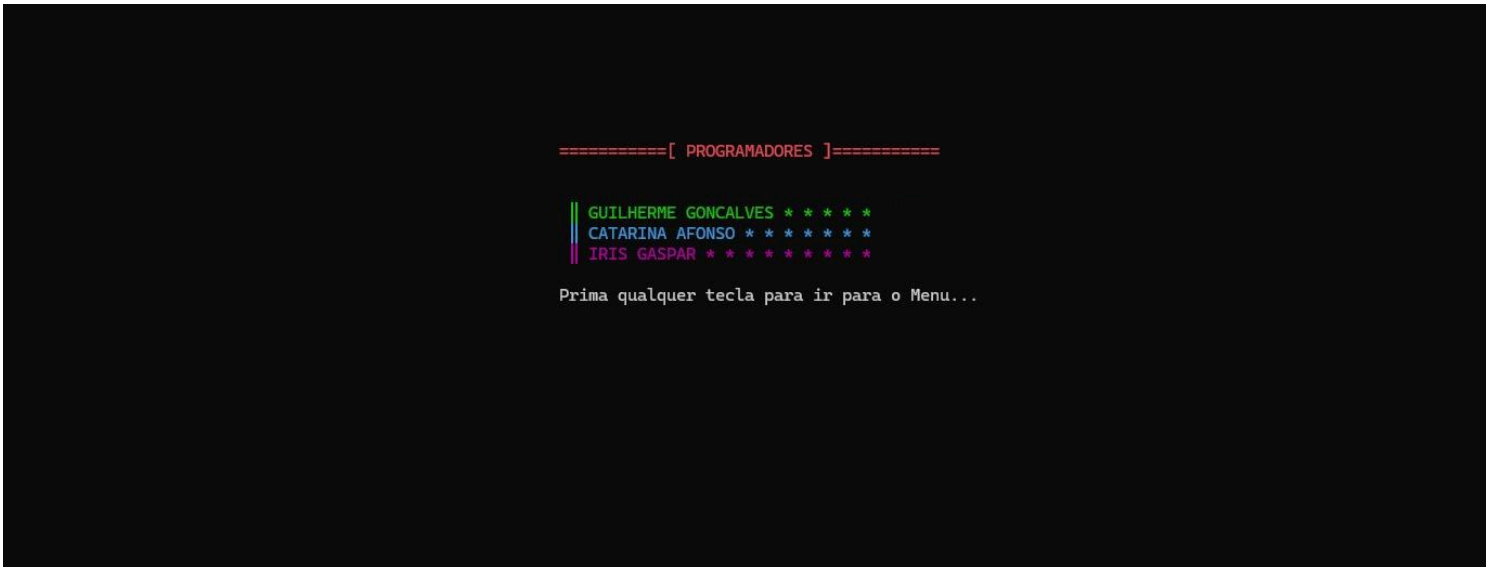


imagem 5.3 => Programadores



imagem 5.4 => GAME OVER

© 2020, DEEC/FCT/UNL

6. Conclusões

A implementação do Jogo da Cobra (Snake Game) em Assembly x64 (MASM) para o ambiente Windows foi concluída com total sucesso, superando os objetivos funcionais iniciais e demonstrando a capacidade de desenvolver um motor de jogo em tempo real utilizando programação de baixo nível. O projeto não só entregou a jogabilidade clássica esperada, mas também incorporou diversas funcionalidades de melhoria.

6.1. Objetivos Cumpridos

Todos os objetivos primários estabelecidos para o projeto foram atingidos:

- **Motor de Jogo Funcional:** A lógica central de movimento (**SpawnHead**), crescimento (**UpdateBody**), e delay temporal (**Sleep**) foi implementada com sucesso, garantindo uma experiência de jogo fluida e estável, validada pelos testes.
- **Colisão e Pontuação:** As verificações da colisão com paredes e auto-colisão são precisas. A pontuação é calculada de forma dinâmica (Tamanho da Cobra * Passos) e exibida corretamente (**DrawScore**).
- **Controlo de Input:** A utilização de **_kbhit e _getch** permitiu um controlo de direção responsivo, essencial para jogos em tempo real.
- **Interface Modular:** O código foi estruturado de forma legível nas três áreas (Interface, Lógica e Input/Sistema), facilitando a manutenção e compreensão.

6.2. Objetivos Extras Cumpridos

Objetivos Extras Atingidos:

- **Persistência do High Score: ShowGameOver** garante que a maior pontuação é guardada (high_score) e exibida nos jogos subsequentes, motivando a repetição do jogo;
- **Estética e Cores: SetConsoleTextAttribute** utiliza cores para destacar a comida, a cabeça da cobra e elementos de arte ASCII (títulos e Game Over), melhorando drasticamente o seu visual;
- **Leitura de Utilizador: ReadUserName** que permite a personalização do jogo e a identificação do jogador no High Score;
- **Efeitos Sonoros**: **Beep** melhora a qualidade de jogo do utilizador, usando efeitos sonoros em situações específicas como o início de jogo, game over e quando a cobra come o alimento.

© 2020, DEEC/FCT/UNL

6.3. Principal Desafio

O principal desafio técnico foi garantir a renderização eficiente e sem piscadelas na consola do Windows, que não foi concebida para atualizações rápidas em tempo real:

- **Problema:** Inicialmente, o uso de funções de I/O lentas ou a repetição desnecessária de redraws causava o piscar dos elementos na grelha.
- **Solução:** A otimização foi alcançada através do uso do **SetConsoleCursorPosition** combinado com **WriteConsoleA** (Windows API) para atualizar apenas os caracteres que mudam (a cabeça, o corpo e o rasto da cauda), minimizando o tempo de atualização por *frame*. O uso de **HideCursor** também contribuiu para a estabilidade visual.

6.4. Objetivo Extra Não Cumprido

O único objetivo ambicioso que não foi totalmente concluído devido a restrições de tempo foi a implementação de um Top 3 de Pontuações.

- **Detalhe:** Embora a lógica para guardar o Highest Score individual (**high_score**) esteja funcional, a expansão para um sistema de ranking que registrasse e ordenasse múltiplos nomes e pontuações (requerendo estruturas de dados mais complexas) não foi finalizada.

6.5. Considerações Finais

Conseguimos cumprir todos os requisitos propostos e ainda acrescentar valor com funcionalidades extra. Embora o objetivo ambicioso de um Top 3 de Pontuações não tenha sido concluído por restrições de tempo, esperemos termos mostrado as nossas capacidades de programação de baixo nível, organização e trabalho de equipa.

© 2020, DEEC/FCT/UNL